



DSA Practice

Tournament bracket (BST)

Content



Introduction



Diagram & Explanation



Programming explain



Demo



Team Members

Keo Kimhong

Team leader

Ngoun Sovichet

Member

Vorng Viseth

Member

Nhean Omra

Member

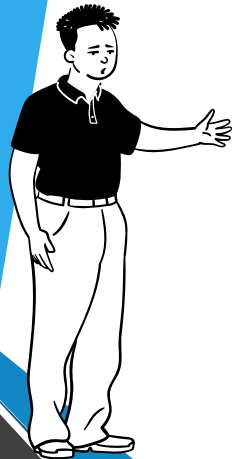
Introduction

What is Binary Search Tree(BST) ?

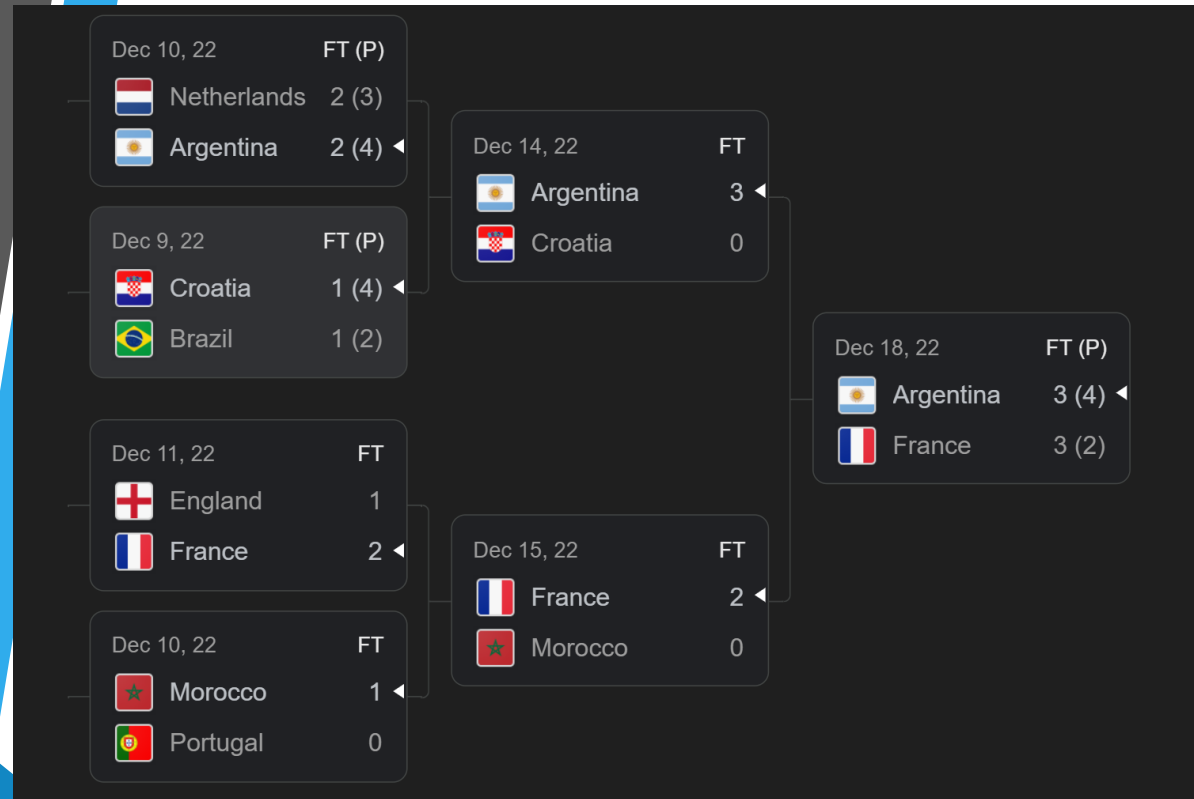
A **Binary Search Tree (BST)** is a special type of binary tree used to store data in a way that makes **searching, inserting, and deleting** efficient.

Why use BST for Tournament bracket ?

The reason why we use **BST** for Tournament bracket is because the **format** of the tournament naturally fits a binary tree root, parents, child, leaf.



Diagram



Child : **The 8 Initial Teams (e.g., Argentina, France, Brazil, etc.)**

Parents : **The Match Node that results in the Semi-Final Winner.**

Root : **The Match Node that determines the Tournament Champion.**

Programming explain

```
class Node{
public:
    Node* left;
    Node* right;
    string name;
    int score;
    int matchId;
    string winner;

    Node(string n , int s, int id){
        left=right=nullptr;
        name=n;
        score=s;
        matchId=id;
        winner="?";
    }
};

class TBracket{
private:
    Node* root;
    queue<Node*> q;
    int matchCounter;
public:
    TBracket(){
        root=nullptr;
        matchCounter=1;
    }
};
```

Programming explain

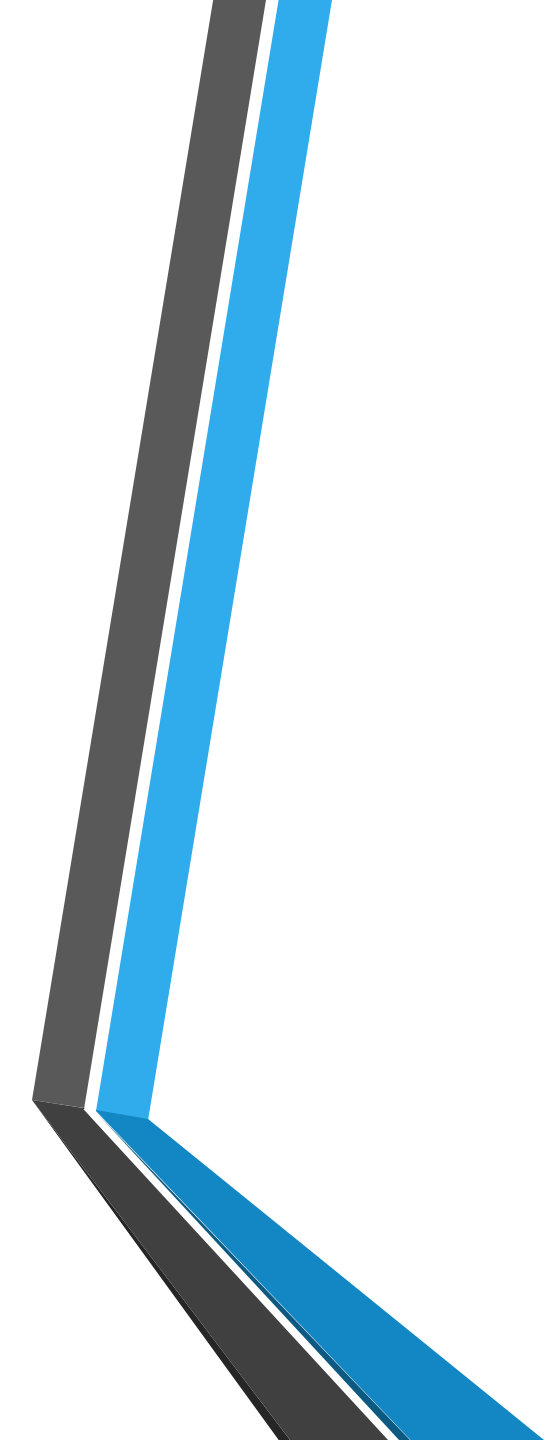
```
vector<string>insert(vector<string> players){ // push every player to
    players=padPlayer(players);
    for(int i=0;i<players.size();i++){
        int randomScore = (players[i]=="BYE"? -1:rand() % 10; // mak
        Node* leaf = new Node(players[i], randomScore,matchCounter++);
        leaf->winner=players[i];
        q.push(leaf);
    }
    return players;
}
```

Programming explain

```
void buildTree(){
    while(q.size()>1){
        Node* A= q.front();
        q.pop();
        Node* B=q.front();
        q.pop();

        Node* match = new Node("Match",0,matchCounter++);
        match->left=A;
        match->right=B;

        if (A->name == "BYE") {
            match->winner = B->winner;
        } else if (B->name == "BYE") {
            match->winner = A->winner;
        } else {
            if(A->score > B->score){
                match->winner= A->winner;
            } else {
                match->winner=B->winner;
            }
        }
        q.push(match);
    }
    root = q.front(); // hold the pointer to every other l
}
```



Demonstration